دليل الاستخدام · الإصدار 0.1

flutter-agent

وكيل محلّي لمعمارية Flutter النظيفة. بحث دلالي داخل كود Dart الخاص بك، وتطبيق صارم لقواعد الطبقات، وتوليد الـ features بأسلوب مشاريعك، ومخزن دروس يتعلّم من جلسات Claude.

يعمل بالكامل على جهازك. لا يخرج أي سطر من الكود إلى الإنترنت، والـ embeddings تُحسب على كرت الشاشة لديك.

المحتوى المفهرس: 896 ملفًا · 1,664 مقطعًا من corereusablepackage وbilhikma وhikmaapp وskygate وattendence.

واجهتان: أدوات MCP داخل Claude Code، وسطر الأوامر `flutter-agent` .

المحتويات

1. **ما هو ولماذا وُجد** — المشكلة التي يحلّها
2. **كيف يعمل** — المسار، وما الذي يُخزَّن
3. **التثبيت** — من الصفر على جهاز جديد
4. **تسجيل مشاريعك** — المصادر والفهرسة الأولى
5. **استخدامه داخل Claude Code** — أدوات MCP
6. **استخدام سطر الأوامر** — كل أمر مع مخرجات حقيقية
7. **توليد feature كامل** — مثال من البداية للنهاية
8. **قواعد المعمارية** — من R1 إلى R12 مع أمثلة
9. **قاعدة المعرفة** — القواعد والأنماط والدروس
10. **حلقة التعلّم** — كيف يتحسّن مع الوقت
11. **حل المشكلات**
12. **مرجع سريع** — المسارات والمتغيّرات والبنية

1. ما هو ولماذا وُجد

أعرافك في كتابة Flutter موجودة في مكانين لا يتحدث أحدهما إلى الآخر: ملف نصّي يشرحها، وقرابة 900 ملف Dart لا يستطيع شيء البحث فيها.

كل جلسة جديدة مع Claude Code تعيد استنتاج معماريتك من `CLAUDE.md` — أي من وصف القواعد، لا من الكود الذي يطبّقها فعلًا. وكل تصحيح تكتبه يضيع بانتهاء الجلسة.

هذا الوكيل يسدّ الفجوتين:

- **يفهرس كودك الحقيقي.** حين تسأل «كيف نعمل pagination؟» يجيبك بـ `PaginatedResult` الخاص بك، لا بمثال عام من الإنترنت.
- **يطبّق قواعدك تطبيقًا ثابتًا.** اثنتا عشرة قاعدة تقرأ الكود وتحكم عليه — بلا أي نموذج ذكاء اصطناعي، فهي سريعة ومجانية ونتيجتها واحدة في كل مرة.
- **يولّد الكود بأسلوبك.** أي feature يولّده مبنيّ على `bilhikma/features/archive`، وهو أنظف تطبيق للمعمارية في مشاريعك.
- **يتذكّر التصحيحات.** حين تصحّح لـ Claude خطأ، يُسجَّل التصحيح ويُفهرس ويعود للظهور في الجلسات القادمة.

ما ليس هو

ليس مولّد كود يغنيك عن التفكير، ولا يقرّر إن كان الكود يُصرَّف (compile) أم لا — فـ `flutter analyze` يبقى المرجع الوحيد في ذلك، والوكيل يستدعيه فعليًا بدل أن يحاكيه.

المعمارية التي يفرضها

كل feature مكوّن من نصفين فقط، ولا وجود لـ `domain/` أو `entities/` أو `use_cases/` أو `repo_interface/`:

```
lib/features/<feature>/
  data/
    data_source/   # the ONLY place that talks to a backend or storage
    models/        # fromJson / toJson
    repos/         # delegation only, no logic
  presentation/
    cubit/         # every decision the screen makes
    refactor/      # body sections pulled out of a screen
    screens/       # 30–70 lines: create the cubit, render the body
    widgets/       # small dumb components
```

2. كيف يعمل



الخطوات

1. **التحليل (parse).** ماسح Dart مكتوب يدويًا يستخرج التعريفات والأعضاء والـ imports والنصوص. ليس مُصرِّفًا ولا يحاول أن يكون — مهمته تحديد الحدود والبيانات الوصفية.
2. **التقطيع (chunk).** مقطع لكل تعريف رئيسي. الـ Cubit الذي يبلغ 600 سطر يُقسَّم عند حدود الدوال، وكل جزء يحتفظ بترويسة الـ class حتى يبقى المقطع مفهومًا بذاته.
3. **الوسم (tag).** كل مقطع يحمل اسم المشروع والـ feature والطبقة والرمز ونطاق الأسطر. هذا ما يجعل `--layer` و`data_source` ممكنًا.
4. **التمثيل (embed).** يُحسب محليًا على كرت RTX 4050 بنموذج `all-MiniLM-L6-v2` (نحو 90 ميغابايت). فهرسة كل الكود تستغرق نحو 24 ثانية.
5. **التخزين.** FAISS للمتجهات وSQLite للبيانات الوصفية، مربوطين بحيث لا يمكن أن يختلفا.
6. **التقديم.** عبر MCP إلى Claude Code، وعبر سطر الأوامر إليك.

> **الفهرسة تراكمية.** لا يُعاد حساب إلا الملفات التي تغيّر محتواها (بمقارنة SHA-256). إعادة الفهرسة بعد تعديل cubit واحد تكلّف عملية واحدة لا تسعمائة — عادةً أقل من ثانيتين.

لماذا التحليل اللغوي مهم

عدّ الأقواس في النص الخام ينهار فور ظهور قوس داخل تعليق أو نص أو داخل `${...}` . لذلك يعمل كل شيء على نسخة *مُقنَّعة* من الملف، تُمحى فيها التعليقات والنصوص حرفًا بحرف مع الحفاظ على المواضع. هذا يتعامل بشكل صحيح مع التعليقات المتداخلة، والنصوص الخام والثلاثية، وحالة `'${map['key']}'` الصحيحة لغويًا والمربكة.

كذلك لا تُعدّ الأقواس المعقوفة إلا خارج الأقواس الهلالية، لأن Dart يكتب الوسائط المسمّاة هكذا: `({required this.repo})` — واعتبارها بداية جسم يقطع كل constructor من هذا النوع إلى نصفين.

مُختبَر على كودك: 885 ملفًا، 1,332 تعريفًا، 8,137 عضوًا، بدون أي انهيار.

3. التثبيت

مُثبَّت بالفعل على هذا الجهاز. هذا القسم موجّه لجهاز جديد أو لزميل في الفريق.

المتطلبات

`Python`	الإصدار 3.10 أو أحدث
`GPU`	اختياري. كرت NVIDIA يسرّع الفهرسة، وبدونه ينتقل تلقائيًا إلى المعالج
`مساحة القرص`	نحو 5 غيغابايت إن أُنشئت بيئة Python جديدة (حزمة PyTorch بـ CUDA تحميلها ~2.8 غيغابايت)
`Flutter`	مطلوب فقط إن أردت التحقق من الكود المولّد بـ `flutter analyze`

التثبيت

```
cd C:\Users\IRD-83\Desktop\local_flutter_agent
.\install.ps1
```

> **المثبّت يعيد استخدام بيئة Python متوافقة موجودة مسبقًا** بدل إنشاء ثانية. على هذا الجهاز يستخدم بيئة `docs-mcp` لأنها تحتوي المكتبات نفسها تمامًا (torch مع CUDA وfaiss وsentence-transformers وmcp)، ولأن نسخة خاصة ستكلّف 4.9 غيغابايت إضافية على قرص ممتلئ بنسبة 91٪.
>
> استخدم `-OwnVenv` إن أردت بيئة مستقلة.

الخيار	أثره
`-OwnVenv`	إنشاء بيئة `.venv` خاصة بدل إعادة الاستخدام (تحميل كبير)
`-Cpu`	تثبيت نسخة PyTorch للمعالج فقط (صغيرة، فهرسة أبطأ)
`-SkipRegister`	عدم تسجيل خادم MCP في Claude Code
`-SkipIndex`	عدم بناء الفهرس في النهاية
`-ReuseVenv <path>`	تحديد ملف `python.exe` بعينه لإعادة استخدامه

إضافة الأمر إلى PATH

الملف التنفيذي داخل البيئة المشتركة، لذا أضف مجلده مرة واحدة:

```
# this session only
$env:PATH += ";C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts"

# permanent
[Environment]::SetEnvironmentVariable("PATH",
  $env:PATH + ";C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts", "User")
```

التسجيل في Claude Code

المثبّت يقوم بذلك. ولعمله يدويًا:

```
claude mcp add --scope user flutter-agent -- `
  C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts\flutter-agent-server.exe
```

ثم **أعد تشغيل Claude Code** ونفّذ `/mcp` ، ويفترض أن ترى `flutter-agent – Connected` .

4. تسجيل مشاريعك

الوكيل يفهرس «مصادر» (source roots): تطبيقات Flutter وحزمًا مشتركة. سجّل الحزمة المشتركة أولًا ثم كل تطبيق.

```
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\corereusablepackage"
flutter-agent sources add "C:\Users\IRD-83\Desktop\my apps\bilhikma"
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\hikmaapp"
flutter-agent sources add "C:\Users\IRD-83\Desktop\my apps\skygate"
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\attendence"

flutter-agent index
```

الفهرس **واحد لكل المشاريع، لا فهرس لكل مشروع** — وهذا مقصود. النمط يجب أن يكون قابلًا للعثور عليه أينما كتبته أول مرة، فالبحث أثناء العمل على `hikmaapp` يجب أن يُظهر الإجابة الصحيحة من `bilhikma` .

ما الذي يُفهرس

كل ملف `.dart` داخل المشروع، عدا الكود المولّد (`*.config.dart` ، `*.gr.dart` ، `*.freezed.dart` ، `*.g.dart`) والاختبارات ومجلدات المنصّات (`*.mocks.dart` ، `web/` ، `ios/` ، `android/` ...). الكود المولّد لا يحمل أي دلالة معمارية وسيُغرق نتائج البحث.

معرفة المسجَّل

```
flutter-agent sources list
```

```
attendence            app      files=111   chunks=224   C:\...\apps\attendence
bilhikma              app      files=379   chunks=624   C:\...\my apps\bilhikma
corereusablepackage   package  files=19    chunks=21    C:\...\apps\corereusablepackage
hikmaapp              app      files=205   chunks=397   C:\...\apps\hikmaapp
skygate               app      files=171   chunks=351   C:\...\my apps\skygate
```

لإزالة مصدر: `flutter-agent sources remove bilhikma` ثم أعد الفهرسة لحذفه من المخزن.

> **الضبط لكل مشروع** اختياري. ضع ملف `.flutter-agent.json` في جذر المشروع لتغيير `include` أو `exclude` أو `max_chunk_chars` أو نموذج الـ embeddings لذلك المشروع وحده.

5. استخدامه داخل Claude Code

هذه الطريقة الأساسية. أنت لا تستدعي الأدوات بنفسك — تطلب طلبك بلغة عادية، وClaude هو من يستدعيها.

ما تقوله	ما يفعله Claude
«أضف feature للإشعارات في bilhikma»	`flutter_pattern` ثم `flutter_component` ثم `flutter_scaffold`
«راجع ما كتبته في features/profile»	`flutter_check` على المجلد
«ملف الـ body هذا طويل، قسّمه»	`flutter_refactor_plan` ثم ينفّذ التقسيم
«كيف نتعامل مع الأخطاء هنا؟»	`flutter_pattern("error handling")`
«هل يوجد widget للحالة الفارغة؟»	`flutter_component()`

ويمكنك تسمية الأداة صراحةً: «استخدم flutter_search لتجد كيف نعمل pagination».

الأدوات العشر

الأداة	ما تجيب عنه
`flutter_pattern`	كيف يفعل هذا الكود كذا — القاعدة ومثال نموذجي والتصحيحات السابقة. **استدعها قبل التنفيذ.**
`flutter_search`	بحث دلالي في كود Dart مع تصفية حسب الطبقة أو الـ feature أو المشروع أو النوع
`flutter_component`	ما تصدّره الحزمة المشتركة، ومع اسم: تعريف المكوّن ومواضع استخدامه الحقيقية
`flutter_check`	مخالفات قواعد المعمارية في ملف أو feature أو مشروع
`flutter_refactor_plan`	ما الذي يُستخرج من ملف متضخّم، مرتّبًا من الأكبر، مع سابقة نظيفة يُحتذى بها
`flutter_scaffold`	مجلد feature كامل مع مفاتيح الترجمة. تجربة بلا كتابة افتراضيًا
`flutter_lesson_add`	تسجيل تصحيح ليُستدعى لاحقًا
`flutter_lessons`	استرجاع التصحيحات حسب الموضوع
`flutter_reindex`	إعادة مسح المصادر بعد تعديلات كثيرة
`flutter_status`	حالة الفهرس والمصادر والجهاز والأعداد

> **لماذا يجعل هذا Claude أرخص كما يجعله أدق.** بلا الوكيل، الإجابة عن «كيف نعمل pagination هنا» تعني قراءة عدة ملفات Dart كبيرة كاملة. أما `flutter_pattern` فيعيد بضع مئات من الـ tokens تحتوي التعريفات ذات الصلة فقط.

6. استخدام سطر الأوامر

كل ما تفعله أدوات MCP متاح مباشرةً. كل المخرجات أدناه حقيقية.

pattern — قبل أن تكتب

```
flutter-agent pattern "error handling"
```

يعيد ثلاثة أشياء دفعةً واحدة: **القاعدة** الحاكمة للموضوع، و**المثال النموذجي** من كودك، وأي **دروس** سُجّلت بشأنه. هذا أنفع أمر في الأداة.

component — قبل أن تكتب widget

```
flutter-agent component
```

```
corereusablepackage exports 18 symbol(s):
  src/theme/app_colors.dart          src/widgets/app_empty_state.dart
  src/theme/app_theme.dart           src/widgets/app_error_state.dart
  src/theme/build_context_ext.dart   src/widgets/app_loading_overlay.dart
  src/widgets/app_button.dart        src/widgets/setting_row.dart
  src/widgets/app_text_field.dart    src/services/toast.dart
  src/widgets/app_card.dart          src/services/app_cache_service.dart
  src/widgets/app_avatar.dart        src/services/screen_security_service.dart
  ...
```

ومع اسم، تحصل على التعريف وكل موضع استُخدم فيه فعلًا:

```
flutter-agent component AppButton
```

search — ابحث عن أي شيء

```
flutter-agent search "firestore query with where filters" --layer data_source
flutter-agent search "how a cubit is wired into a screen" --layer screen -n 3
flutter-agent search "pagination" --project bilhikma --paths-only
```

الخيار	معناه
`-n, --limit`	عدد النتائج (افتراضيًا 5)
`--layer`	`core`، `widget`، `screen`، `refactor`، `cubit`، `repo`، `model`، `data_source`
`--feature`	اسم مجلد الـ feature، مثل `archive`
`--project`	اسم المشروع، مثل `bilhikma`
`--kind`	`lesson`، `pattern`، `rule`، `code`
`--paths-only`	المواضع فقط دون نص الكود

check — بعد أن تكتب

```
flutter-agent check "C:\Users\IRD-83\Desktop\my apps\bilhikma\lib\features\home"
```

```
package=bilhikma  1 error(s), 2 warning(s)

lib/features/home/presentation/refactor/home_screen_body.dart:194
    [R3 error] refactor body is 194 lines, over the 150-line ceiling
    fix: split the body into presentation/refactor/ or presentation/widgets/
lib/features/home/presentation/screens/home_screen.dart:81
    [R3 warning] screen is 81 lines, over the 70-line target
```

النطاق مهم. فحص feature واحد يعرض مخالفات ذلك الـ feature فقط. أما فحص `lib/` كاملًا فيضيف تدقيق الترجمة على مستوى المشروع (القاعدة R8) — وهناك يظهر الاختلاف بين LangKeys وar.json وen.json. في bilhikma هناك 130 مفتاح ترجمة غير معرّف في LangKeys.

refactor — خطة تقسيم

```
flutter-agent refactor "C:\...\features\home\presentation\refactor\home_screen_body.dart"
```

```
lib/features/home/presentation/refactor/home_screen_body.dart
  layer: refactor body   lines: 194 (target 120, ceiling 150)
  rules: 1 error(s), 0 warning(s)
    R3 L194: refactor body is 194 lines, over the 150-line ceiling

  split candidates, largest first:
    _HomeSections.build       L101-192 (92 lines) -> presentation/widgets/
    HomeScreenBody.build      L39-83  (45 lines) -> presentation/refactor/
```

الـ classes الخاصة (التي تبدأ بـ `_`) تُوجَّه إلى `widgets/` والعامة إلى `refactor/`. والأمر يعيد خطة لا تعديلًا، لأن إعادة الهيكلة قرار يحتاج حكمًا بشريًا.

أوامر الصيانة

`flutter-agent index`	إعادة مسح كل المصادر (تراكمية؛ أضف `--force` لإعادة البناء كاملًا)
`flutter-agent status`	حالة الفهرس والجهاز والأعداد حسب النوع والطبقة
`flutter-agent doctor`	التأكد من أن كرت الشاشة يُستخدم
`flutter-agent sources list`	المشاريع المسجّلة
`flutter-agent export-dataset out.jsonl`	تصدير الدروس كبيانات تدريب

7. توليد feature كامل

يولّد ثمانية ملفات مع ربط الترجمة، على نسق `bilhikma/features/archive` .

الخطوة 1 — تجربة بلا كتابة

```
flutter-agent scaffold notifications --root "C:\Users\IRD-83\Desktop\my apps\bilhikma"
```

```
files:
  [new] lib/features/notifications/data/data_source/notifications_data_source.dart
  [new] lib/features/notifications/data/models/notifications_model.dart
  [new] lib/features/notifications/data/repos/notifications_repo.dart
  [new] lib/features/notifications/presentation/cubit/notifications_cubit.dart
  [new] lib/features/notifications/presentation/cubit/notifications_state.dart
  [new] lib/features/notifications/presentation/refactor/notifications_body.dart
  [new] lib/features/notifications/presentation/screens/notifications_screen.dart
  [new] lib/features/notifications/presentation/widgets/notifications_tile.dart

edits:
  [update] lib/core/localization/lang_keys.dart   - add 4 constant(s)
  [update] assets/translations/ar.json            - add 4 key(s)
  [update] assets/translations/en.json            - add 4 key(s)

localization keys:
  LangKeys.notificationsTitle      = 'notifications_title'
  LangKeys.notificationsEmpty      = 'notifications_empty'
  LangKeys.notificationsError      = 'notifications_error'
  LangKeys.notificationsLoadFailed = 'notifications_load_failed'
```

التجربة تطبع محتوى كل ملف كاملًا، ولا تكتب شيئًا على القرص.

الخطوة 2 — التنفيذ

```
flutter-agent scaffold notifications --root "C:\...\bilhikma" --write
```

ويرفض الكتابة فوق ملفات موجودة إلا بإضافة `--force` .

الخطوة 3 — التحقق

```
cd "C:\Users\IRD-83\Desktop\my apps\bilhikma"
flutter analyze lib/features/notifications
```

`No issues found!` — الكود المولّد يُصرَّف كما هو. وهذا مُختبَر: المولّد نفسه مطلوب منه أن يجتاز قواعد الوكيل الاثنتي عشرة.

الخطوة 4 — ما يتبقّى عليك

الـ data source يأتي بهيكل عامل مع عقد الأخطاء جاهزًا. استبدل جسم `fetchItems` بالاستدعاء الحقيقي:

```
Future<List<NotificationsModel>> fetchItems() async {
  try {
    // replace this with the real backend call
    const List<Map<String, dynamic>> raw = <Map<String, dynamic>>[];
    return raw.map(NotificationsModel.fromJson).toList(growable: false);
  } on NotificationsException {
    rethrow;
  } catch (error) {
    throw NotificationsException(mapError(error));
  }
}
```

ثم سجّله في الـ service locator، ولاحظ `registerFactory` للـ cubit:

```
getIt
  ..registerFactory(() => NotificationsCubit(repo: getIt()))
  ..registerLazySingleton(() => NotificationsRepo(dataSource: getIt()))
  ..registerLazySingleton(NotificationsDataSource.new);
```

> **الـ cubits تُسجَّل دائمًا بـ `registerFactory`.** تسجيل cubit كـ singleton يبقي حالة الشاشة السابقة ويُطلق `emit` بعد `close()` — وهو أشهر سبب لمشكلة «الحالة من آخر مرة فتحت فيها هذه الشاشة».

التسمية

اسم الـ feature يُطبَّع تلقائيًا، فـ `LessonDetail` و`lesson-detail` و`lesson detail` كلها تعطي `lesson_detail` و`LessonDetail` و`lessonDetail` كلٌّ في موضعه الصحيح.

8. قواعد المعمارية

اثنتا عشرة قاعدة ثابتة. لا نموذج ذكاء اصطناعي في أيٍّ منها — كل قاعدة خاصية في الكود يمكن الحكم عليها بالقراءة.

كل المطابقات تجري على النسخة المُقنَّعة من الملف، لذا **يستحيل** أن تنطلق قاعدة بسبب كلمة داخل تعليق أو نص.

الرمز	القاعدة	الشدّة
R1	بنية المجلدات — لا `domain/` ولا `entities/` ولا `use_cases/` ولا `repo_interface/` ولا `repo_impl/` ولا `presentations/`، مع أسماء مجلدات معيارية	خطأ / تحذير
R2	ممنوع استيراد المشروع من نفسه بـ `package:<own_project>/…` داخل `lib/`	خطأ
R3	الملفات ضمن حدّ الأسطر المسموح لطبقتها	خطأ / تحذير
R4	الشاشات لا تحتوي دوال `_build…`	خطأ
R5	طبقة العرض لا تستدعي Firestore أو Supabase أو SharedPreferences أو Dio أو http أو Hive أو التخزين الآمن	خطأ
R6	الـ cubits لا تمسّ `BuildContext` ولا `Navigator` ولا `ScaffoldMessenger` ولا الحوارات	خطأ
R7	نصوص الواجهة تمرّ عبر `LangKeys` — لا `'raw_key'.tr()` ولا نص مكتوب مباشرةً	خطأ / تحذير
R8	`LangKeys` و `ar.json` و `en.json` تحمل المفاتيح نفسها	خطأ / تحذير
R9	الـ repos تمرّر بلا أي منطق تفرّع	تحذير
R10	لا `Expanded` / `Flexible` داخل منطقة تمرير غير محدودة الارتفاع (قاعدة استدلالية)	تحذير
R11	الـ models تعرّف `fromJson` أو `toJson` على الأقل	تحذير
R12	كل feature يحوي المجلدات المعيارية (مجلدات التجميع مستثناة)	تحذير

حدود الأسطر (R3)

الطبقة	المستهدف	الحد الأقصى
Screen	70	150
Refactor body / widget	120	150
Repo	120	250
Model	200	400
Cubit / data source	400	900

الـ cubit له حدّ واسع لأن السلوك وظيفته فعلًا. أما الشاشة فحدّها ضيّق لأن الشاشة التي تجاوزته تكون قد ابتلعت عملًا يخصّ cubit أو widget.

القاعدة R6 عمليًا — أهم قاعدة

```
// WRONG - the cubit reaches into the widget tree
Future<void> save() async {
  await repo.save();
  Navigator.of(context).pop();
  ScaffoldMessenger.of(context).showSnackBar(...);
}

// RIGHT - the cubit emits, the screen listens
Future<void> save() async {
  await repo.save();
  if (isClosed) return;
  emit(state.copyWith(status: ProfileStatus.saved));
}
```

الـ cubit يعيش أطول من أي widget وقد يُصدر حالة بعد اختفائه. الاحتفاظ بـ context ينتج عنه في أحسن الأحوال خطأ «استخدام BuildContext عبر فجوة غير متزامنة»، وفي أسوئها انهيار التطبيق.

> `if (isClosed) return;` **بعد كل** `await` **وقبل كل** `emit`**.** المستخدم قد يغادر الشاشة أثناء الطلب، والإصدار على cubit مغلق يرمي استثناءً. هذا السطر موجود في كل cubit في كودك — ليس أسلوبًا اختياريًا.

القاعدة R7 عمليًا

```
Text('saved_items'.tr())   // raw key - nothing stops a typo
Text('Saved items')        // hardcoded, never translated
Text(LangKeys.savedItems.tr())
```

الثابت ليس زينة: `'saved_itmes'.tr()` يُصرَّف ويُنشر ويعرض المفتاح نفسه للمستخدم، بينما `LangKeys.savedItmes` لا يُصرَّف أصلًا.

والنصوص المُدمَجة وغير اللغوية تُستثنى بشكل صحيح: `Text('$count')` و `Text('-')` و `Text('2x')` .

ما وجده في مشاريعك

المشروع	النتيجة
`corereusablepackage`	**صفر مخالفات** — المرجع النظيف يجتاز الفحص
`bilhikma`	10 أخطاء: 3 استيرادات ذاتية وملفات تجاوزت الحد؛ و130 مفتاح ترجمة غير معرّف
`skygate`	585 خطأ: 436 استيرادًا ذاتيًا و177 نص ترجمة خامًا و26 ملفًا تحت `views/` و `utils/`

9. قاعدة المعرفة

مجلد `knowledge/` ملفات Markdown عادية تقرأها وتعدّلها وتضعها في Git. نفّذ `flutter-agent index` بعد أي تعديل.

المجلد	النوع	المحتوى
`rules/`	`rule`	لماذا وُجد القيد، مع مثال قبل/بعد حقيقي
`patterns/`	`pattern`	أمثلة نموذجية مأخوذة من كودك مع شرح القرارات
`lessons/lessons.jsonl`	`lesson`	تصحيحات تُضاف تباعًا من الجلسات

الموجود حاليًا

بطاقات قواعد: file-size، errors، localization، layering.
بطاقات أنماط: dependency-injection، pagination، data-source-and-model، screen-and-body، cubit-and-state.

> **هذا هو الجزء الذي تنمّيه أنت.** أي موضوع بلا بطاقة ستكون نتيجته أضعف. وأسرع طريقة لإضافة بطاقة أن تطلب من Claude: «اكتب بطاقة نمط لرفع الصور بناءً على طريقة skygate» — فيقرأ الكود ويكتب الملف، ثم تعيد الفهرسة.

كيف تكتب بطاقة جيدة

- **اعرض كودًا حقيقيًا** من مشاريعك، لا أمثلة مخترعة.
- **اشرح القرار لا الصياغة.** «لماذا وُجد `clearError`» مفيد، أما «copyWith ينسخ الحالة» فلا.
- **أدرج النسخة الخاطئة.** البحث يصبح أدق كثيرًا في التقاط خطأ على وشك الحدوث حين يكون الخطأ مكتوبًا.
- **موضوع واحد لكل ملف** بعناوين `##` — فالمقطّع يعتمد عليها.

10. حلقة التعلّم

هذا ما يجعله وكيلًا يتحسّن، لا مجرّد فهرس بحث جامد.

متى تسجّل درسًا

1. **حين تصحّح كود Flutter الذي كتبه Claude.** سجّل ما كُتب وما كان يجب أن يُكتب والسبب.
2. **حين يرفض `flutter analyze` كودًا مولّدًا.** سجّل الخطأ وإصلاحه.
3. **حين يجد `flutter_check` مخالفة ثم تُصلَح.** مرّر رمز القاعدة بـ `--rule` ليظهر الدرس بجوارها.
4. **حين يتبيّن عُرف لا تغطّيه أي قاعدة.** إن تكرّر، رقِّه إلى بطاقة في `rules/` .

تسجيل درس

```
flutter-agent lesson "cubit registered as singleton" `
  "getIt..registerFactory(() => ArchiveCubit(repo: getIt()));" `
  --wrong "getIt..registerLazySingleton(() => ArchiveCubit(repo: getIt()));" `
  --why "A singleton cubit keeps the previous screen's state and emits after close()." `
  --tag di --tag cubit --layer cubit --rule R6
```

`Recorded lesson 9e014a3bcf6a and indexed: cubit registered as singleton`

وداخل Claude Code لا تكتب هذا بنفسك — تقول «احفظ هذا» بعد التصحيح، فيستدعي Claude أداة `flutter_lesson_add` .

الاسترجاع

```
flutter-agent lessons "how do I register a cubit in get_it"
```

والدروس تظهر تلقائيًا أيضًا ضمن نتائج `flutter-agent pattern` ، وهذا هو المقصود: أن يظهر التصحيح بجوار النمط الذي يصحّحه، قبل تكرار الخطأ.

> **اجعل حقل `right` ملموسًا.** درس تصحيحه «تذكّر أن تستخدم الـ repo» لا قيمة له، أما درس يحتوي الأسطر الخمسة الصحيحة فعلًا فله قيمة.

الدروس كبيانات تدريب

```
flutter-agent export-dataset lessons.jsonl
```

يحوّل كل درس إلى ثلاثية تعليمات/استجابة (`system` / `user` / `assistant`). هذا هو الطريق المؤجَّل لتدريب نموذج محلي على أسلوبك — ويستحق إعادة النظر عند بضع مئات من الدروس الحقيقية، لا قبل ذلك.

عن الـ fine-tuning بصراحة

على كرت شاشة محمول بسعة 6 غيغابايت، تدريب QLoRA على ملفات Dart الطويلة بطيء وعائده متواضع: التدريب يرسّخ الأسلوب لا الصحة، وسيظل النموذج يخترع دوال غير موجودة. أما البحث الدلالي مع محرّك القواعد فيعطيك معظم الفائدة بجزء يسير من التكلفة. البيانات هي الأصل الحقيقي؛ اجمعها واتخذ القرار لاحقًا.

11. حل المشكلات

العَرَض	السبب والحل
الأدوات لا تظهر في Claude Code	أعد تشغيل Claude Code بعد التسجيل ثم نفّذ `/mcp` . وتحقّق بـ `claude mcp list` .
`flutter-agent: command not found`	المجلد غير مضاف إلى PATH — راجع القسم 3، أو استدعِ ملف `.exe` بمساره الكامل.
البحث لا يعيد شيئًا	نفّذ `flutter-agent status` . إن كان عدد المقاطع صفرًا فلا مصادر مسجّلة أو مفهرسة.
الفهرسة بطيئة أو تظهر `device: cpu`	نفّذ `flutter-agent doctor` . إن كان CUDA غير متاح فقد ثُبّتت نسخة PyTorch الخاصة بالمعالج.
نفاد ذاكرة كرت الشاشة	برنامج آخر يحجز الذاكرة (ComfyUI سبب شائع — كان يستهلك 5.0 من 6.1 غيغابايت على هذا الجهاز). أغلقه أو اضبط `FLUTTER_AGENT_DEVICE=cpu` .
`health: BROKEN`	اختلاف بين FAISS وSQLite. نفّذ `flutter-agent index --force` .
النتائج قديمة	نفّذ `flutter-agent index` . الفهرسة تراكمية وتستغرق ثانية أو ثانيتين عادةً.
الفحص يغرق في تحذيرات R8	وجّهته إلى جذر المشروع. وجّهه إلى مجلد feature واحد بدلًا من ذلك.
قاعدة تنطلق خطأً	القاعدة R10 استدلالية صراحةً وتقول ذلك. أما البقية فهذا خلل — القواعد مُختبَرة بحالات صحيحة وخاطئة في `tests/test_rules.py` .

التحقق من سلامة التثبيت كاملًا

```
flutter-agent doctor      # is the GPU in use?
flutter-agent status      # index healthy, counts sensible?
flutter-agent sources list
pytest tests -q           # 53 passed
```

12. مرجع سريع

أين توجد الأشياء

`C:\Users\IRD-83\.flutter-agent\index`	فهرس FAISS وبيانات SQLite
`C:\Users\IRD-83\.flutter-agent\sources.json`	المصادر المسجّلة
`\local_flutter_agent\knowledge\…`	بطاقات القواعد والأنماط والدروس
`\local_flutter_agent\src\flutter_agent\…`	الحزمة نفسها
`\local-mcp\.venv\Scripts\…`	بيئة Python المشتركة والملفات التنفيذية

متغيّرات البيئة

`FLUTTER_AGENT_HOME`	موضع الفهرس (افتراضيًا `~/.flutter-agent`)
`FLUTTER_AGENT_KNOWLEDGE`	مجلد المعرفة (افتراضيًا `knowledge/` داخل المشروع)
`FLUTTER_AGENT_MODEL`	نموذج الـ embeddings (افتراضيًا `all-MiniLM-L6-v2`)
`FLUTTER_AGENT_DEVICE`	`cuda` أو `cpu` (افتراضيًا: كشف تلقائي)

بنية الكود

`config.py`	المسارات والقيم الافتراضية ومطابقة الأنماط
`sources.py`	سجلّ مشاريع Flutter المفهرسة
`embedder.py`	sentence-transformers على CUDA مع التراجع إلى المعالج
`store.py`	FAISS وSQLite والأعمدة الوصفية والبحث المُصفّى
`dart_parser.py`	محلل التقنيع وماسح الأقواس
`dart_chunker.py`	تقطيع واعٍ بالتعريفات
`layers.py`	تحويل المسار إلى (مشروع، feature، طبقة)
`rules.py`	القواعد من R1 إلى R12
`scaffold.py`	مولّد الـ features وربط الترجمة
`lessons.py`	حلقة التعلّم وتصدير البيانات
`indexer.py`	فهرسة تراكمية متعدّدة المصادر
`service.py`	تنسيق المخرجات ضمن حدّ محدّد
`server.py`	خادم MCP
`cli.py`	أمر `flutter-agent`

الدورة اليومية مختصرة

```
# before writing
flutter-agent pattern "<topic>"
flutter-agent component

# while writing
flutter-agent scaffold <feature> --root <project>          # dry run
flutter-agent scaffold <feature> --root <project> --write

# after writing
flutter-agent check <feature folder>
flutter analyze

# when corrected
flutter-agent lesson "<topic>" "<correct code>" --wrong "..." --why "..."

# housekeeping
flutter-agent index
```

---

flutter-agent الإصدار 0.1 — 5,274 سطر Python و53 اختبارًا. أُنشئ في 1 أكتوبر 2026.